

# Classificação de câncer de pele usando Transfer Learning

dataset font: [k Skin Cancer MNIST: HAM10000](#)

Eu fiz uma função para download usando a Classificação da Kaggle. Minha preferência para o modelo pré treinado foi o **EfficientNet-B4**. Podia ter escolhido o B0 ou o B2, mas analisei os parâmetros desse modelo, para a classificação o trabalho de profundidade, largura e resolução foi o maior peso na minha escolha e a acurácia normalmente alta, esse modelo exigiu muito mais processamento da minha máquina, demorou para treinar, mas obtive resultado que achei bom. Fiz o treinamento em duas etapas, uma de transfer learning 'head' com backbone congelado e apenas a última camada com as 7 classes e outra usando fine-tuning.

Abaixo, apresento a explicação detalhada de cada componente, organizada por funcionalidade:

## 1. Configurações Iniciais e Utilidades

- **set\_seed(seed)**: Garante a reprodutibilidade dos experimentos. Fixa a semente para as bibliotecas random, numpy e torch (CPU e GPU), para que os resultados sejam os mesmos em diferentes execuções.
- **load\_dotenv\_if\_exists(env\_path)**: Um carregador manual de variáveis de ambiente. Procura um arquivo .env para ler as credenciais do Kaggle sem precisar da biblioteca python-dotenv.
- **configure\_kaggle\_environment()**: Configura as chaves de API necessárias para baixar o dataset do Kaggle. Ela verifica se as credenciais estão no ambiente ou no arquivo padrão ~/.kaggle/kaggle.json.
- **resolve\_device(device\_arg)**: Detecta automaticamente o melhor hardware disponível para processamento: **CUDA**(NVIDIA), **MPS** (Apple Silicon) ou **CPU**.
- **build\_data\_loader\_kwargs(args, device)**: Otimiza o carregamento de dados. Se estiver usando GPU, ativa o pin\_memory(que acelera a transferência de dados para a placa de vídeo).

## 2. Gerenciamento de Dados

### Classe SplitFrames

Uma @dataclass simples (basicamente uma estrutura de dados) usada apenas para organizar os três subconjuntos de dados (treino, validação e teste) após a divisão do DataFrame original.

## Classe HAM10000Dataset

Herda de `torch.utils.data.Dataset`. É o "mapa" que o PyTorch usa para acessar as imagens:

- **`__len__`**: Retorna o número total de imagens.
- **`__getitem__`**: Quando o modelo pede uma imagem, este método abre o arquivo .jpg, converte para RGB, aplica as transformações (aumentos de dados) e retorna a imagem com seu respectivo rótulo.

### Funções de Processamento

- **`ensure_dataset(root_dir)`**: Verifica se o dataset existe localmente. Se não existir, aciona a API do Kaggle para baixá-lo e descompactá-lo. Retorna um mapeamento de IDs de imagem para caminhos de arquivo.
- **`find_metadata_file(root_dir)`**: Uma função auxiliar que busca recursivamente pelo arquivo CSV de metadados.
- **`build_splits(...)`**: Divide o dataset em Treino, Validação e Teste. Utiliza a técnica de **estratificação** (stratify), garantindo que a proporção de cada tipo de doença seja a mesma em todos os grupos.
- **Meu split foi de:**

**treino: 70%**

**validacao: 15%**

**teste: 15%**

- **`make_transforms(image_size)`**: Define o pré-processamento.
  - **Treino**: Inclui *Data Augmentation* (rotação, inversão horizontal e brilho) para evitar overfitting.
  - **Validação/Teste**: Apenas redimensiona e normaliza as imagens.

## 3. Arquitetura do Modelo

- **`build_model(num_classes, freeze_backbone)`**:
  1. Carrega a **EfficientNet-B4** pré-treinada na ImageNet.
  2. **`freeze_backbone`**: Se verdadeiro, congela as camadas iniciais (as que já sabem identificar formas e cores).
  3. Substitui a "cabeça" (camada final) original por uma nova camada linear ajustada para o número de classes do HAM10000 (geralmente 7).
- **`unfreeze_backbone(model)`**: Libera todas as camadas do modelo para que os pesos possam ser ajustados durante o Fine-tuning.

## 4. Ciclo de Treinamento e Avaliação

- **run\_epoch(...)**: O núcleo do processamento. Passa todas as imagens do DataLoader pelo modelo.
  - Se for treino, calcula o erro (loss) e atualiza os pesos (backpropagation).
  - Coleta as previsões e probabilidades para calcular métricas depois.
- **compute\_metrics(...)**: Calcula uma bateria de métricas de performance: Acurácia, F1-Score (macro e ponderado), Precisão, Recall e a curva ROC AUC.
- **evaluate\_split(...)**: Uma função de conveniência que roda run\_epoch em modo de avaliação e gera um relatório detalhado (classification\_report) e uma matriz de confusão.

## 5. Orquestração (O fluxo principal)

Executa os seguintes passos:

1. Prepara o ambiente e o dataset.
2. Cria os DataLoaders.
3. **Etapa Head (Transfer Learning)**: Treina apenas a nova camada final por 5 épocas com um *Learning Rate* maior.
4. **Etapa Finetune**: Descongela o modelo todo e treina com um *Learning Rate* muito pequeno para ajustar os detalhes finos às imagens médicas.
5. Salva o melhor modelo baseado no F1-Score da validação.
6. Gera relatórios finais em CSV e JSON.

Resumo do Fluxo de Trabalho

O código segue a estratégia de **Transfer Learning**:

1. **Congelar** a base -> Treinar a cabeça (ajuste rápido).
2. **Descongelar** tudo -> Treinar levemente (ajuste fino).

Eu penso que como as bordas, cores e formas e outros parâmetros foram treinados em datasets maiores, essa técnica melhora a performance e a possível falta de dados robustos.

**Resultados:**

### **1. Evolução do Treinamento (Loss e F1-Score)**

## Fase [head] (Épocas 1-5)

Nesta fase, os pesos da EfficientNet estavam congelados e você treinou apenas a nova camada de saída.

- **Comportamento:** A perda (`val_loss`) caiu de 0.80 para 0.69.
- **Conclusão:** É um aquecimento necessário. O modelo aprendeu a mapear as características gerais da ImageNet para as classes de lesões de pele, mas o F1-score baixo (~0.35) indica que ele ainda não era capaz de distinguir bem as classes mais sutis.

## Fase [finetuning] (Épocas 1-10)

Camadas descongeladas, permitiu que o modelo ajustasse seus filtros internos para imagens médicas.

- **Salto de Performance:** Logo na primeira época de finetuning, o `val_f1` saltou de 0.34 para 0.44.
- **Estabilidade:** A perda de validação continuou caindo até a Época 10 (0.4677), mostrando que o modelo não sofreu um *overfitting* severo imediato.
- **Ponto de Atenção:** Na Época 8, houve um pequeno aumento na `val_loss` (0.49), que depois voltou a cair. Isso sugere que o *learning rate* está em um bom patamar, permitindo que o modelo escape de mínimos locais.

## 2. Análise das Métricas Finais (Test Set)

O conjunto de teste é a prova real. Com **83% de Accuracy**, o modelo parece ótimo, mas o segredo está nas outras métricas:

### Accuracy (83.03%) vs. Balanced Accuracy (61.48%)

Essa diferença é o ponto mais importante da sua análise.

- O dataset HAM10000 é altamente desbalanceado (muitas imagens de "Melanocytic nevi" e poucas de "Dermatofibroma" ou "Melanoma").
- A Accuracy de 83% pode ser enganosa: se o modelo chutar a classe mais comum o tempo todo, ele terá uma acurácia alta.
- A Balanced Accuracy de 61.48% é a média da sensibilidade de cada classe. Ela revela que o modelo tem dificuldade em identificar as classes minoritárias (as doenças mais raras).

### F1-Macro (0.6485) vs. F1-Weighted (0.8134)

- **F1-Weighted:** Dá mais peso às classes com mais fotos. Como o modelo vai bem nas classes comuns, essa métrica fica alta.
- **F1-Macro:** Trata todas as classes como iguais (raras ou comuns). Ter 0.64 aqui indica que o modelo está fazendo um trabalho honesto, mas ainda erra consideravelmente

em algumas categorias específicas (provavelmente as malignas, que costumam ter menos exemplos).

ROC-AUC Macro (0.9637)

Este valor é excelente. Ele indica que, embora o modelo possa errar a classificação final (threshold), ele consegue separar muito bem as probabilidades das classes. Ou seja, a "confiança" do modelo está bem calibrada na maioria dos casos.

3. Comparação: Treino vs. Validação vs. Teste

| Métrica  | Treino | Validação | Teste  |
|----------|--------|-----------|--------|
| Loss     | 0.3914 | 0.4676    | 0.4692 |
| Accuracy | 85.81% | 82.96%    | 83.03% |
| F1-Macro | 0.6927 | 0.6558    | 0.6485 |

Exportar para Sheets

**Diagnóstico:** O modelo tem um **overfitting leve**. A diferença entre o F1-Macro de treino (0.69) e teste (0.64) é pequena (~5%). Isso indica que as técnicas de regularização e o Transfer Learning funcionaram bem e o modelo generaliza satisfatoriamente para dados novos.

Professor Bruno para a avaliação das métricas e explicação dos métodos pedi ajuda para uma I.A, para melhorar minha percepção dos resultados e explicar melhor cada um desses resultados.

Posso melhorar um pouco mais o modelo alterando alguns parâmetros, mas achei esses resultados satisfatórios para a aplicação de tranfer learning nessa atividade. Muito obrigado.